

sgNetApi - Especificación Técnica

Documentación de Integración: Módulo de Administración, Seguridad y Auditoría

Framework: .NET 10 Web API (Clean Architecture)

Versión API: v1.0.0

Seguridad: JWT Bearer + RBAC Granular

Fecha de Emisión: 20 de Agosto de 2026

1. Visión General de la Arquitectura de Seguridad

El backend **sgNetApi** implementa una arquitectura basada en Clean Architecture estructurada en tres capas (Domain, Infraestructure y Api). El Módulo de Administración y Seguridad proporciona control de acceso basado en el patrón **RBAC** (Role-Based Access Control) acoplado con permisos granulares por acción.

Principio de Separación de Responsabilidades (RBAC Granular)

Los **Permisos** son constantes técnicas declaradas por el equipo de desarrollo en los controladores C#. Los **Roles** son agrupadores dinámicos gestionados por usuarios administradores desde la interfaz web. La autorización se evalúa dinámicamente mediante el atributo `[RequirePermission("nombre.permiso")]`.

Petición HTTP Client → Bearer JWT → AuditAndExceptionMiddleware → PermissionAuthorizationHandler → Controller → PostgreSQL

2. Esquema de Autenticación y Formato del Token JWT

La autenticación se realiza mediante cadenas compuestas JWT (JSON Web Tokens) firmadas simétricamente con algoritmos HMAC-SHA256. Todo request a endpoints protegidos debe adjuntar la cabecera estándar de autorización.

Authorization: Bearer <tu_token_jwt_aqui>

Estructura de Claims dentro del JWT:

Claim Key	Tipo	Descripción / Ejemplo
sub / nameid	String	Cédula de Identidad del usuario autenticado (ej: "43791806").
email	String	Correo institucional asociado (ej: "wilson.arriola@sgnet.com.uy").
role	String (Multiple)	Roles asignados al usuario (ej: "Administrador", "Operador").
permiso	String (Multiple)	Colección consolidada de permisos directos y heredados por roles (ej: "admin.usuarios.leer").

3. Matriz Completa de Permisos del Sistema

La siguiente tabla detalla los permisos estándar instalados en el sistema por el `DataSeeder`:

ID	Nombre del Permiso	Descripción Técnica	Controlador / Endpoint Afectado
1	<code>admin.usuarios.crear</code>	Permite registrar nuevos usuarios en el sistema.	<code>POST /api/Usuarios</code>
2	<code>admin.usuarios.leer</code>	Consulta de listado general y detalle individual de usuarios.	<code>GET /api/Usuarios</code> , <code>GET /api/Usuarios/{ci}</code>
3	<code>admin.usuarios.editar</code>	Modificación de datos personales, asignación de roles y reseteo de clave.	<code>PUT /api/Usuarios/{ci}</code> , <code>PUT /api/Usuarios/{ci}/roles-permisos</code>
4	<code>admin.usuarios.desbloquear</code>	Cambio de estado (habilitado/deshabilitado) y desbloqueo por intentos fallidos.	<code>PATCH /api/Usuarios/{ci}/estado</code>
5	<code>roles.administrar</code>	Consulta del catálogo de permisos, creación y modificación de roles.	<code>GET /api/Permisos</code> , <code>GET /api/Roles</code> , <code>POST/PUT /api/Roles</code>
6	<code>admin.auditoria.leer</code>	Consulta paginada con filtros del historial de logs HTTP.	<code>GET /api/Auditoria</code>
7	<code>admin.auditoria.exportar</code>	Descarga del reporte de auditoría en formato CSV (UTF-8 con BOM).	<code>GET /api/Auditoria/exportar</code>

4. Especificación Completa de Endpoints de la API

4.1 Módulo Autenticación y Perfil (`/api/Auth`)

POST `/api/Auth/login`

Descripción: Inicia sesión, valida clave y retorna el token JWT. Incrementa contador de intentos fallidos y bloquea al 3er fallo.

```
{
  "ci": 43791806,
  "password": "Admin.123456"
}
```

POST `/api/Auth/cambiar-password`

Descripción: Cambia la clave del usuario autenticado. Valida que no coincida con las últimas 5 claves en `HistorialPassword`.

4.2 Módulo Gestión de Usuarios (`/api/Usuarios`)

GET `/api/Usuarios` | **GET** `/api/Usuarios/{ci}`

Descripción: Obtiene la lista o el detalle completo de un usuario por CI. Exige permiso `admin.usuarios.leer`.

PUT `/api/Usuarios/{ci}/roles-permisos`

Descripción: Actualización atómica de roles y permisos directos. Exige permiso `admin.usuarios.editar`.

```
{
  "idsRoles": [1, 2],
  "idsPermisosDirectos": [6]
}
```

4.3 Módulo Roles y Permisos (/api/Roles y /api/Permisos)

GET /api/Permisos

Descripción: Retorna el catálogo completo de permisos disponibles para armar la interfaz en el frontend. Exige roles.administrar.

POST /api/Roles | **PUT** /api/Roles/{idRol}

Descripción: Crea o actualiza un Rol y sincroniza en lote su colección de permisos. Exige roles.administrar.

```
{
  "nombre": "Supervisor de Operaciones",
  "idsPermisos": [1, 2, 6]
}
```

4.4 Módulo Auditoría y Logs (/api/Auditoria)

GET /api/Auditoria

Descripción: Consulta paginada y filtrable de la bitácora de auditoría HTTP. Exige admin.auditoria.leer.

```
Parámetros Query: ?pagina=1@istrosPorPagina=10&usuarioCi=43791806&codigoEstado=200
```

GET /api/Auditoria/exportar

Descripción: Genera y descarga directamente un archivo .csv con codificación UTF-8 BOM apto para Excel. Exige admin.auditoria.exportar.

5. Formato Estándar de Errores y Excepciones

El AuditAndExceptionHandler intercepta los errores imprevistos (500) y las fallas de validación devolviendo estructuras JSON homogéneas:

Código HTTP	Causa	Estructura de Respuesta JSON
401 Unauthorized	Token JWT ausente, expirado o malformado.	{ "mensaje": "No autorizado." }
403 Forbidden	Token válido pero carece del permiso exigido.	{ "mensaje": "Acceso denegado. Requiere el permiso '...'" }
500 Internal Error	Excepción no controlada en servidor.	{ "codigo": 500, "mensaje": "Ocurrió un error interno en el servidor. El incidente ha sido registrado.", "fecha": "2026-08-20T10:54:27Z" }